

Prompt 5 — Contrato de lectura con Métrica (F2)

Paralelizable con los prompts 4 y 6.

Repo de especificación: github.com/leanchoi/leanchoi, rama `claude/esquel-data-ecosystem-integration-qbxuv2`.
Leé `AGENTS.md` (sobre todo I1) y `docs/03-pipeline-datos.md` §2.

REGLA ABSOLUTA: Métrica no se modifica. Ni una línea de código, ni un contenedor, ni una tabla, vista, índice o función dentro de su esquema. Si tu plan incluye una vista materializada dentro de Métrica, el plan está mal – se evaluó y se descartó (`AGENTS.md` §6).

PASO 1 – Descubrir el esquema real

`specs/sql/02_metrica_contract.sql` usa nombres SUPUESTOS (`price_observations`, `listings`, `destinations`, `fx_daily`). Conectate y listá `information_schema.tables`. Mapeá los reales.
CORREGÍ EL ARCHIVO DE SPEC en el mismo commit.

PASO 2 – Rol de solo lectura

```
CREATE ROLE esquel_ro LOGIN PASSWORD 'pwd';
GRANT CONNECT ON DATABASE metrica TO esquel_ro;
GRANT USAGE ON SCHEMA public TO esquel_ro;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO esquel_ro;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO esquel_ro;
ALTER ROLE esquel_ro SET statement_timeout = '120s';
ALTER ROLE esquel_ro SET idle_in_transaction_session_timeout = '60s';
```

Crear un rol es un cambio de base, no de la aplicación: no toca tablas, datos ni el despliegue del puerto 3013. Es MENOS riesgoso que consultar con las credenciales de la app, porque hace imposible una escritura accidental.

PASO 3 – Contrato versionado

<code>etl/contracts/metrica/ota_daily.sql</code>	destino x noche x tipología	(~29k filas/año)
<code>etl/contracts/metrica/ota_leadtime.sql</code>	destino x noche x bucket	(~36k filas/año)
<code>etl/contracts/metrica/fx.sql</code>	tipo de cambio diario	
<code>etl/sources_metrica.py</code>	ejecutor	

Detalles que importan:

- `application_name='esquel_data_etl'` en la conexión, para identificarlo en `pg_stat_activity`.
- Extracción FUERA de la ventana de scraping de Métrica.
- `DISTINCT ON (listing_id, noche)` tomando la observación MÁS RECIENTE. Sin esto, los listings muestreados más veces pesan más en la mediana y el ADR queda sesgado hacia las propiedades que el scraper visitó más.
- `HAVING` con mínimo muestral, coherente con las reglas de indicadores.py.
- La "ocupación implícita" NO es ocupación real: nombrala así en todo el pipeline para que nadie la confunda con la del OIT ni con la EOH.

PASO 4 – Medir antes de optimizar

Medí el p95 de la extracción completa durante 30 corridas. Si supera 10 s, evaluá materializar DEL LADO DE ESQUEL DATA (una tabla DuckDB local), nunca dentro de Métrica. Con ~360k observaciones no debería hacer falta: son agregaciones subsegundo.

CRITERIOS DE ACEPTACIÓN

1. esquel_ro NO puede escribir: un INSERT de prueba falla con permission denied. Verificalo explícitamente y dejá constancia.
2. p95 < 10 s en 30 corridas.
3. Con Métrica detenida, el ETL continúa y marca el dataset desactualizado. NO aborta.
4. pg_stat_activity muestra application_name='esquel_data_etl' durante la extracción.
5. Los agregados cuadran contra un conteo manual sobre las tablas base.
6. specs/sql/02_metrica_contract.sql corregido con los nombres reales.
7. `docker ps` y `git status` en /root/scrapper/metrica: sin cambios de ningún tipo.